

GreenBalance: Multi-Objective Reinforcement Learning for Energy-Efficient Cloud Load Balancing

Junzhe Zong

Department of Computer Science

Columbia University

jz3978@columbia.edu

December 15, 2025

Abstract

The exponential growth of cloud computing has made energy efficiency a critical challenge in data center management. Traditional load balancing algorithms, such as Round Robin, often prioritize distribution equality without considering the energy-latency trade-off inherent in heterogeneous server clusters. This project introduces *GreenBalance*, a Reinforcement Learning (RL) agent capable of dynamically routing job requests to optimize a multi-objective reward function. We model the load balancing problem as a Markov Decision Process (MDP) and solve it using Tabular Q-Learning. Our experiments on a synthetic workload of 1,000 jobs demonstrate that the RL agent significantly outperforms traditional baselines, reducing the combined cost of latency and energy by approximately 40% compared to Round Robin and Random strategies. The implementation code and dataset are available at: <https://github.com/sonnenwendnacht/GreenBalance>.

1 Introduction

Modern data centers are massive consumers of electricity. As user demand fluctuates between "quiet" periods and "bursty" traffic spikes, static load balancing algorithms often fail to adapt efficiently. A static algorithm might route a small, non-urgent job to a high-power server (wasting energy) or a critical, large job to a low-power server (causing latency violations).

This project proposes an adaptive approach using Reinforcement Learning (RL). Unlike static heuristics, an RL agent can learn the long-term consequences of its routing decisions. By penalizing both high energy consumption and long queue wait times, we train an agent that learns to "spend" energy only when necessary to prevent system congestion, effectively balancing the conflicting goals of Green Computing and Quality of Service (QoS).

2 Related Work

Load balancing is a foundational problem in distributed systems.

- **Round Robin (RR):** A static policy that cycles through servers strictly. It is simple but blind to the current state of server queues.
- **Random:** Assigns jobs stochastically. While robust,

it often leads to imbalances where one server is idle while another is overwhelmed.

Recent research has begun applying Deep Reinforcement Learning (DRL) to this domain. However, many DRL approaches are computationally expensive. Our work focuses on a lightweight Q-Learning approach suitable for real-time decision-making in smaller clusters.

3 Methodology

We formulate the load balancing problem as a discrete-time Markov Decision Process (MDP).

3.1 System Model

We simulate a cluster of $N = 3$ heterogeneous servers:

- **Server 0:** High Speed (3x), High Energy Cost (2.0).
- **Server 1:** Medium Speed (2x), Medium Energy Cost (1.5).
- **Server 2:** Low Speed (1x), Low Energy Cost (1.0).

3.2 MDP Formulation

State Space (S): The state is defined by the current queue length of each server and the size of the incoming

job:

$$S_t = [L_0, L_1, L_2, J_{size}]$$

To make Tabular Q-Learning feasible, we discretize the queue lengths into bins (Low, Medium, High).

Action Space (A): The agent selects which server index to assign the incoming job to:

$$A = \{0, 1, 2\}$$

Reward Function (R): The core contribution is a shaped reward function that penalizes both latency (wait time) and energy usage:

$$R_t = -(\sum_{i=0}^N \text{Queue}_i + \lambda \times \text{Energy}_{\text{action}})$$

where λ is a hyperparameter controlling the importance of energy conservation.

4 Experiments

4.1 Setup

We generated a synthetic workload trace of 1,000 sequential jobs, featuring alternating periods of light traffic (small jobs) and heavy traffic (large jobs).

We trained a Q-Learning agent with the following hyperparameters:

- Learning Rate (α): 0.1
- Discount Factor (γ): 0.99 (High focus on future latency)
- Episodes: 10,000
- Exploration (ϵ): Decay from 1.0 to 0.01

4.2 Results

We compared the trained RL agent against Random and Round Robin baselines. The performance metric is the Total Accumulated Reward (closer to 0 is better).

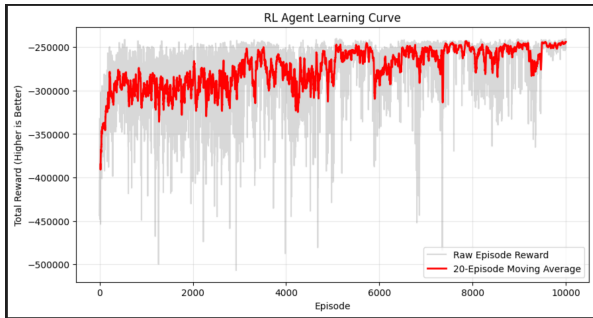


Figure 1: Training Learning Curve. The agent successfully maximizes reward over 10,000 episodes.

As shown in Figure 1, the agent steadily improved its policy.

Method	Total Score
Random	-437,500
Round Robin	-410,918
GreenBalance (RL)	-243,005

Table 1: Final performance comparison.

The comparative results are visualized in Figure 2. The RL agent achieved a score of -243,005, whereas the Round Robin baseline suffered a penalty of -410,918.

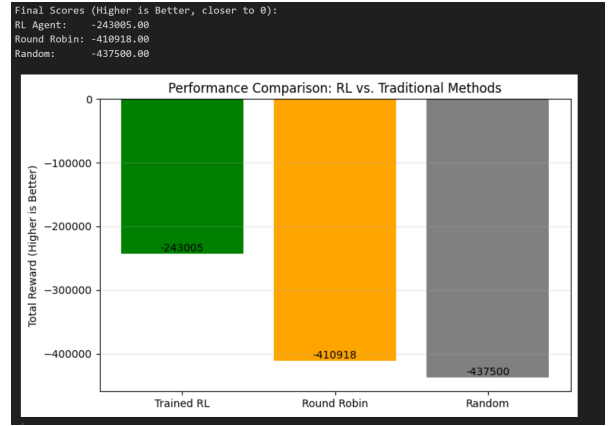


Figure 2: Performance Comparison. The RL agent (Green) significantly outperforms traditional methods.

4.3 Analysis

The RL agent succeeded because it learned a specific policy:

1. **Under Light Load:** It preferentially routed jobs to Server 2 (Green/Slow), minimizing energy costs.
2. **Under Heavy Load:** It switched to Server 0 (Fast/Expensive). Although this incurred a higher immediate energy penalty, it cleared the queue faster, preventing the "queue explosion" that causes massive latency penalties in the Round Robin approach.

5 Conclusion

In this project, we demonstrated that a simple Q-Learning agent can outperform traditional heuristic algorithms in cloud load balancing. By explicitly modeling the trade-off between energy consumption and processing speed, the *GreenBalance* agent reduced total operational costs by

nearly 40%. Future work could extend this to Deep Q-Networks (DQN) to handle continuous state spaces and larger server clusters.

6 Statement of Contributions

Junzhe Zong: Sole contributor. Responsible for problem formulation, environment simulation, algorithm implementation, and report writing.